



UNIVERSIDAD AUTÓNOMA DE CHIHUAHUA
FACULTAD DE INGENIERÍA
INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN
Bases de Datos Avanzadas



4.24 Proyecto Final

Grupo: 7CC2

Nombre del Alumno:
Joseph Daniel Ramírez Torres
374352

Docente:
JOSE SAUL DE LIRA MIRAMONTES

Chihuahua, Chih. 26/05/26

Descripción General

Este proyecto implementa una aplicación web desarrollada con el microframework **Flask** (Python) y la base de datos de grafos Neo4j. Permite gestionar nodos de tipo *Department* y *Employee* mediante operaciones CRUD completas.

Las relaciones modeladas en el grafo son:

- (Employee)-[:WORKS_IN]->(:Department) — el empleado pertenece a un departamento
- (Employee)-[:REPORTS_TO]->(:Employee) — jerarquía de jefe/subordinado

Link de GitHub

<https://github.com/CourRam/proyecto-3-BDavanzadas-neo4j>

Estructura del Proyecto

El código se organizó aplicando el principio de separación de responsabilidades: cada archivo tiene una única función. Esto facilita el mantenimiento, las pruebas y el trabajo en equipo.

Archivo	Responsabilidad
config.py	Variables de entorno: URI, usuario y contraseña de Neo4j
db.py	Driver de Neo4j + helpers run_query() / run_write()
app.py	Punto de entrada: crea la app Flask y registra los Blueprints
routes/departments.py	Blueprint /departments — CRUD completo de departamentos
routes/employees.py	Blueprint /employees — CRUD completo de empleados
routes	Blueprint— vista HTML

Código Documentado

config.py

Lee las credenciales de conexión a Neo4j desde variables de entorno del sistema operativo. Si la variable no existe, usa el valor por defecto. Centralizar la configuración aquí permite cambiar el entorno (desarrollo, producción) sin modificar el código de la aplicación.

db.py

Crea una única instancia del driver de Neo4j que se reutiliza en toda la aplicación (patrón singleton implícito).

Expone dos funciones que abstraen el ciclo abrir-sesión / ejecutar / cerrar-sesión:

- **run_query()** — ejecuta consultas de lectura (MATCH ... RETURN) y devuelve una lista de diccionarios con los registros.
- **run_write()** — ejecuta escrituras (CREATE / MERGE / SET / DELETE) y no devuelve nada (None).

app.py

Punto de entrada de la aplicación. Su única responsabilidad es instanciar Flask y registrar los tres Blueprints.

La ruta raíz / muestra un resumen con el conteo de nodos en la base de datos.

routes/departments.py

Blueprint con prefijo /departments. Gestiona el ciclo de vida completo del nodo :Department. Captura la excepción ConstraintError de Neo4j cuando se intenta crear un departamento con un número ya existente, mostrando un mensaje de error en lugar de colapsar la aplicación. La variable readonly controla si el campo de número de departamento es editable en el formulario.

routes/employees.py

Blueprint con prefijo /employees. Además del CRUD, gestiona dos relaciones del grafo. Se extrajeron funciones privadas (prefijo _) para evitar duplicación entre create() y edit():

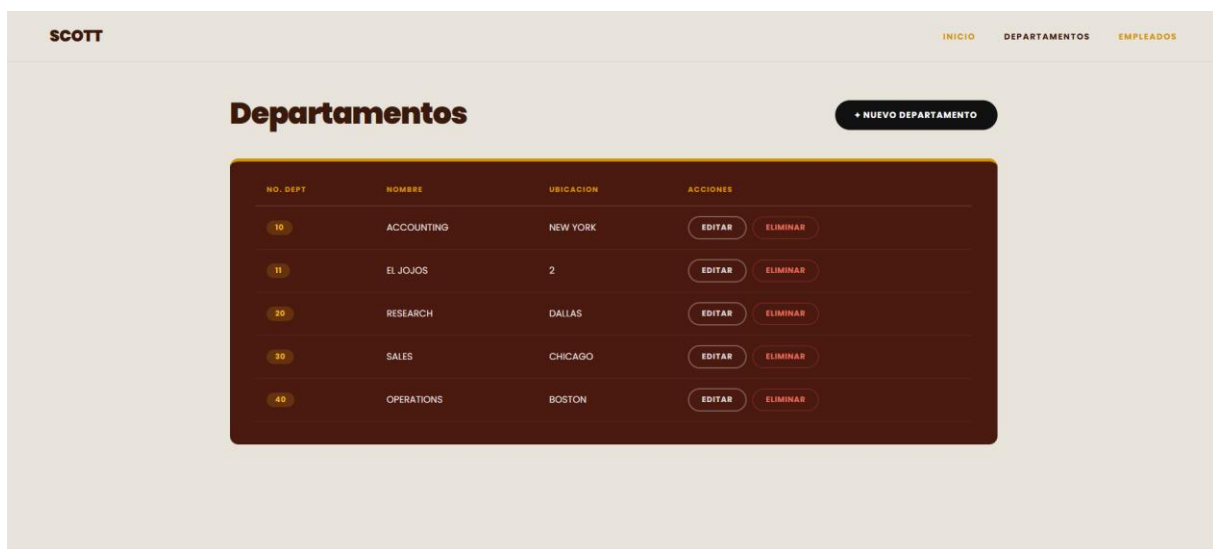
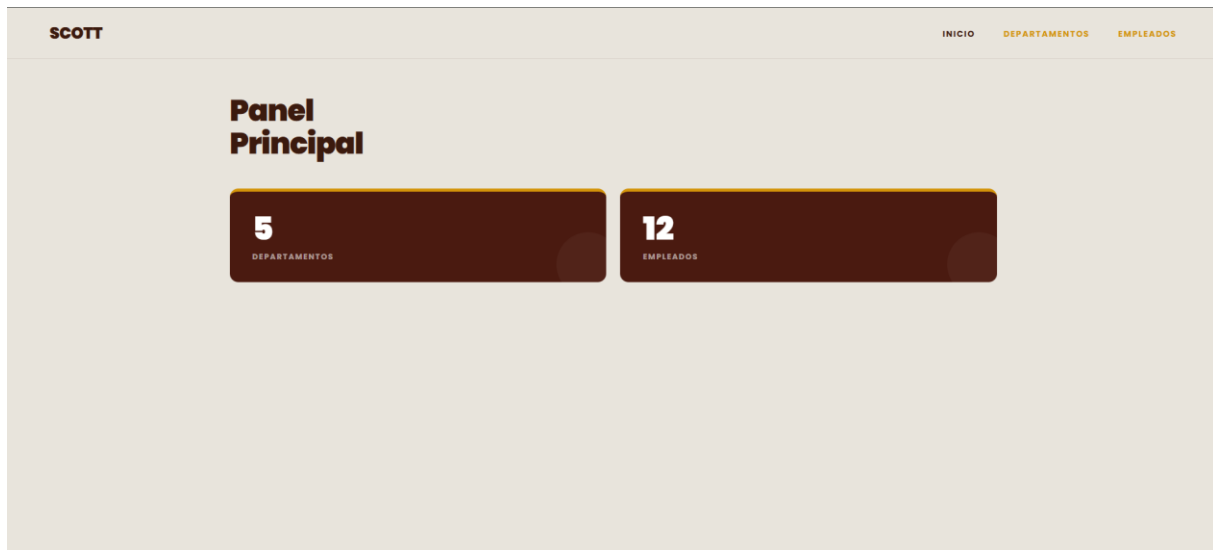
- **_get_departments()** — consulta los departamentos para poblar el select del formulario.
- **_create_works_in(empno, deptno)** — elimina la relación WORKS_IN previa y crea la nueva.
- **_create_reports_to(empno, mgr)** — elimina la relación REPORTS_TO previa y, si hay jefe, crea la nueva.

Manejo de Errores

Se implementó manejo explícito del error de unicidad de Neo4j (ConstraintError) tanto en departamentos como en empleados. Cuando el usuario intenta crear un registro con un ID ya existente, la aplicación no colapsa: captura la excepción, conserva los datos escritos en el formulario y muestra un mensaje descriptivo

La variable readonly se pasa independientemente de dept/emp, para que el campo de ID siga editable durante la creación (incluso cuando hay error) y sea de solo lectura únicamente al editar un registro existente.

Capturas de Pantalla



SCOTT

INICIODEPARTAMENTOSEMPLEADOS

Crear Departamento

CANCELAR

DATOS DEL DEPARTAMENTO

NUMERO DE DEPARTAMENTO

Ej. 50

NOMBRE

Ej. ACCOUNTING

UBICACION

Ej. NEW YORK

CREAR

CANCELAR

SCOTT

INICIODEPARTAMENTOSEMPLEADOS

Empleados

+ NUEVO EMPLEADO

NO. EMP	NOMBRE	PUESTO	SALARIO	COMISION	FECHA ALTA	JEFE	DEPARTAMENTO	ACCIONES
1	JOSEPH	CHINSON	\$3.00	—	2001-10-15	—	EL JOJOS	EDITAR ELIMINAR
7821	WARD	SALESMAN	\$1250.00	\$500.00	1981-02-22	7698	SALES	EDITAR ELIMINAR
7866	JONES	MANAGER	\$2975.00	—	1981-04-02	7839	RESEARCH	EDITAR ELIMINAR
7864	MARTIN	SALESMAN	\$1250.00	\$1400.00	1981-09-28	7698	SALES	EDITAR ELIMINAR
7868	BLAKE	MANAGER	\$2850.00	—	1981-05-01	7839	SALES	EDITAR ELIMINAR
7792	CLARK	MANAGER	\$2450.00	—	1981-06-09	7839	ACCOUNTING	EDITAR ELIMINAR
7788	SCOTT	ANALYST	\$3000.00	—	1987-04-19	7566	RESEARCH	EDITAR ELIMINAR
7839	KING	PRESIDENT	\$5000.00	—	1981-11-17	—	ACCOUNTING	EDITAR ELIMINAR

SCOTT

INICIODEPARTAMENTOSEMPLEADOS

Crear Empleado

CANCELAR

Nodo Employee

DATOS DEL EMPLEADO

NUMERO DE EMPLEADO

Ej. 7999

NOMBRE

Ej. SMITH

PUESTO

Ej. CLERK

NO. JEFE (OPCIONAL)

Dejar vacio si no aplica

FECHA DE ALTA

dd/mm/aaaa

SALARIO

Ej. 2000.00

COMISION (OPCIONAL)

Dejar vacio si no aplica

DEPARTAMENTO

Seleccionar...

CREAR

CANCELAR